

Slot - D

23

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
 Department of Artificial Intelligence and Data Science  
**ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING**

|                  |                                                                                      |               |                                                |
|------------------|--------------------------------------------------------------------------------------|---------------|------------------------------------------------|
| Course Code:     | CSA1407                                                                              | Course Title: | COMPILER DESIGN                                |
| CO Assessed:     | CO1 - Imitation (L1)                                                                 | Assessment:   | Assessment Tool 1 - ANALYTICAL PROBLEM SOLVING |
| Weightage:       | 40% of CIA                                                                           |               |                                                |
| CO1 Statement:   | Apply lexical analysis for token specification and recognition using LEX tool. (BL3) |               |                                                |
| Student Name:    | M.S. Gowrish Shankar                                                                 | Reg. No.:     | 192424270                                      |
| Section / Batch: | B.Tech AI & DS                                                                       | Date:         | 16/07/2026                                     |

**Code of Conduct**

I, M.S. Gowrish (192424270) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.S. Gubli-Robdy.

**Instructions**

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

| Q. No                        | Problem Understanding (20) | Method Selection (20) | Solution Process (25) | Accuracy of Calculation (20) | Interpretation of Results (15) | Total Score (out of 100) |
|------------------------------|----------------------------|-----------------------|-----------------------|------------------------------|--------------------------------|--------------------------|
| 1                            | 4 / 4                      | 3 / 4                 | 4 / 4                 | 4 / 4                        | 3 / 4                          | 91.25                    |
| 2                            | 4 / 4                      | 3 / 4                 | 3 / 4                 | 4 / 4                        | 4 / 4                          | 86.75                    |
| 3                            | 3 / 4                      | 4 / 4                 | 3 / 4                 | 4 / 4                        | 3 / 4                          | 85                       |
| 4                            | 4 / 4                      | 4 / 4                 | 3 / 4                 | 3 / 4                        | 3 / 4                          | 85                       |
| 5                            | 3 / 4                      | 3 / 4                 | 4 / 4                 | 4 / 4                        | 4 / 4                          | 90                       |
| <b>OVERALL RUBRICS VALUE</b> |                            |                       |                       |                              |                                |                          |
|                              | / 4                        | / 4                   | / 4                   | / 4                          | / 4                            | 88                       |
|                              | / 20                       | / 20                  | / 25                  | / 20                         | / 15                           | / 100                    |

*N.D.A*



## Annexure A

### ANALYTICAL PROBLEM SOLVING

- |          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b> | <p>Trace the processing of the input expression <math>P = (a * b) + (b * c) * 2</math> through all phases of a compiler.</p> <p>a) Identify and list all tokens generated during lexical analysis.</p> <p>b) Construct the syntax tree during the parsing phase.</p> <p>c) Generate the intermediate code for the given expression.</p> <p>d) Apply suitable optimizations to the intermediate code.</p> <p>e) Explain the significance of the final target code prod</p>                                                |
| <b>2</b> | <p>For each of the following Regular Expressions, construct the language.<br/>Write any FIVE valid strings for each Regular Expression.</p> <p>a) <math>(\epsilon + aa^*)^*</math></p> <p>b) <math>(a^* + b^*)^* abb</math></p> <p>c) <math>(a^*b^*)^* aba (a^* + b^*)^*</math></p> <p>d) <math>(a+ab)^*</math></p> <p>e) <math>(aba)^* + (a^*b^*)^*</math></p>                                                                                                                                                          |
| <b>3</b> | <p>i) Find the number of tokens in the following C statement is</p> <pre>main( ) { int a=10; char b ="abc"; in t c = 30; ch ar d ="xyz"; in /* comment */ t m = 40.5; }</pre> <p>ii) Identify the lexical errors in the following C statement is</p> <pre>void main() {     int x=10, y=20;     char * a;     a= &amp;x;     x= 1xab; }</pre>                                                                                                                                                                            |
| <b>4</b> | <p><b>Find a regular expression corresponding to each of the following subsets of <math>\{0, 1\}^*</math>:</b></p> <p>i) The language of all strings that have an even number of 0's.</p> <p>ii) The language of all strings that contain at least one 1.</p> <p>iii) The language of all strings in which both the number of 0's and the number of 1's are odd.</p> <p>iv) The language of all strings that start with 1 and end with 0.</p> <p>v) The language of all strings that do not contain consecutive 1's.</p> |

Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, \*, /, =, <, >
3. Separators: (, ), {, }, ., ;
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

```
int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }
```

### Questions:

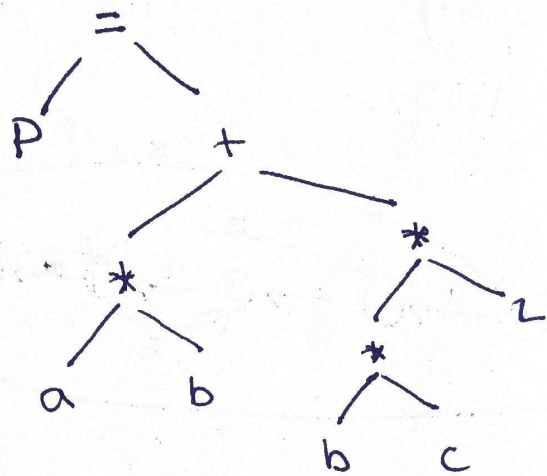
- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.



1). Trace the Processing of the Expression  
 $P = (a * b) + (b * c) * 2$

(a). Tokens Generated during lexical analysis  
Total tokens = 15

(b) Syntax tree:



(c) Intermediate Code:

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

(d) Optimized Code:

$$t_1 = a * b$$

$$t_2 = b * c$$

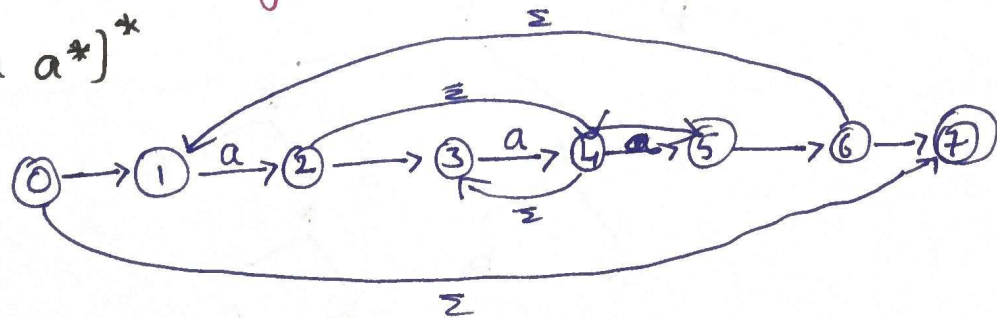
$$P = t_1 + t_2 * 2$$

(c) Significance of final target code:

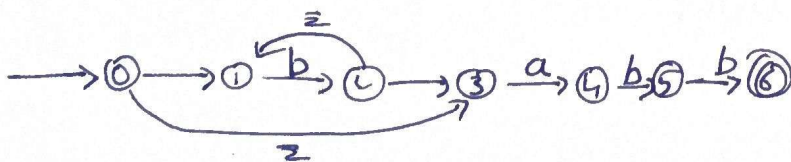
- \* Machine dependent code executed by the processor.
- \* Optimised for speed and memory.
- \* Produces efficient executable programs.

2). Regular expressions - write any five valid strings.

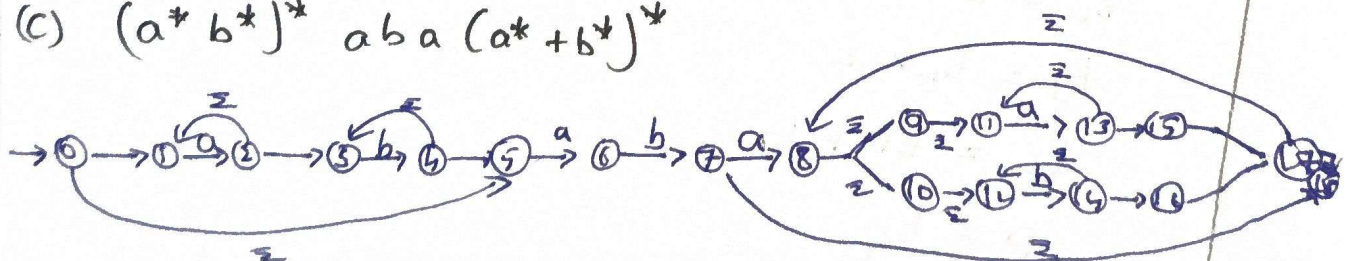
(a)  $(a a^*)^*$



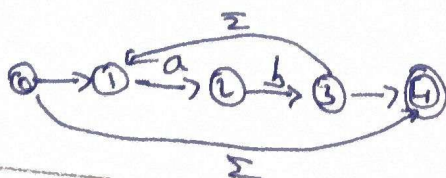
(b)  $(b^*)^* a b b$



(c)  $(a^* b^*)^* a b a (a^* + b^*)^*$

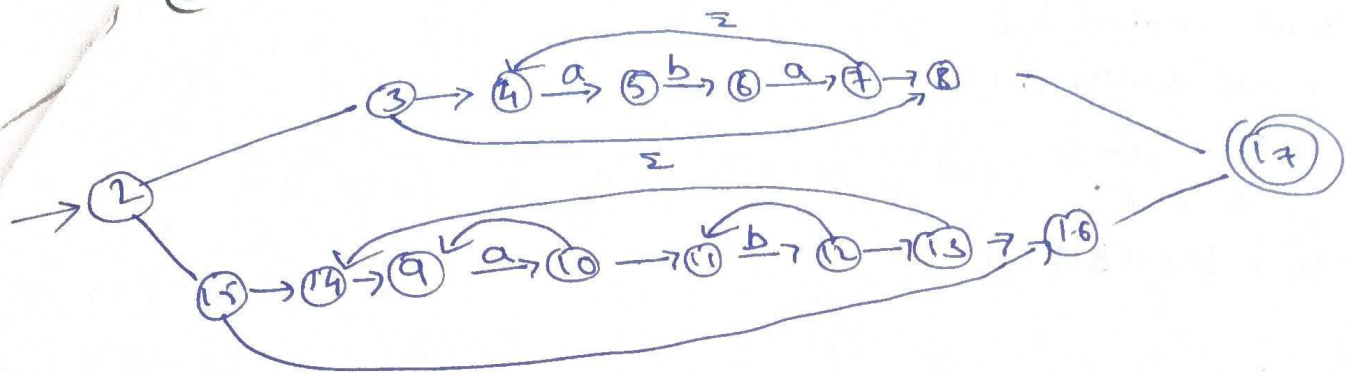


(d)  $(a b)^*$





$(aba)^* + (a^*b^*)^*$



### 3). (i) Number of Tokens

```
main() {
    int a = 10;
    char b = "abc";
    int c = 30;
    char d = "xyz";
    int m = 40.5; }
```

| <u>Token</u>               | <u>Type</u>        | <u>(identification count)</u> |
|----------------------------|--------------------|-------------------------------|
| main                       | identifier         | 1                             |
| ( )                        | separator          | 2                             |
| { }                        | separator          | 2                             |
| int, char                  | Keywords           | 5                             |
| a, b, c, d, m              | identifiers        | 5                             |
| ;                          | separator          | 5                             |
| =                          | operator           | 5                             |
| 10, "abc", 30, "xyz", 40.5 | Strings, constants | 5                             |

Total tokens = 30

## (ii) Lexical Errors

```
void main() {  
    int x=10, y=20;  
    char *a;  
    a = "x";  
    x = 1xab; }  
}
```

Errors:

The error is 1xab

It considers 1 as number & xab as constant  
as lexical error.

4. Find the regular expression corresponding to each of the following subsets of  $\{1\}^*$ :

(a) Even number of 0's

Regular expression

$1^*(01^*01^*)^*$

(b) Atleast one 1

$(0+1)^*1(0+1)^*$

(c) Both number of 0's and 1's are odd

$(11)^*(00)^*1(11)^*0((11)^*(00)^*)$

(d) starts with 1 and ends with 0

$1(0+1)^*0$

(e) NO consecutive 1's

$0^*(0+01)^*1?$

5. Lexical  
int



## Source Code Analyser Problem

```
int var123 = 12345;  
float var1236 = 123.45;  
if (var123 >= var1236) {  
    return;  
}
```

(1) Number of tokens

Total number of tokens = 20.

(2) Categorized Tokens

Key words (4)

- int
- float
- if
- return

operators (3)

- =
- >
- >=

separators (7)

- ;
- ;
- (
- )
- {
- ;
- }



### Identifiers (4)

- var 123
- var 456
- var 123
- var 456

### Integer literals (1)

- 12345

### Floating-point Literals (1)

- 123.45

### 3. Estimated FSM state Transitions

| Token Type       | count | Avg. Transition | Total |
|------------------|-------|-----------------|-------|
| key words        | 4     | 5               | 20    |
| Identifiers      | 4     | 7               | 28    |
| operators        | 3     | 2               | 6     |
| separators       | 7     | 1               | 7     |
| Integer literal  | 1     | 5               | 5     |
| Floating literal | 1     | 7               | 7     |

Estimated total FSM state Transitions  $\approx 73$ .



# CRITERIA FOR CALCULATION:

| Criteria                  | Weightage | Level 4 – Exemplary                                                               | Level 3 – Proficient                      | Level 2 – Developing                       | Level 1 – Beginning                           |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Problem understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/for formula with strong justification             | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |